

Security Assessment of Mobile Application Attack Surface

& Comprehensive Exploitation Scenarios

NAME	SHAIK THOUHID
COURSE CODE	CSA5802
COURSE NAME	DEVSECOPS
REG NO	192421280

1. Executive Summary

This case study provides an in-depth security assessment of 'FinShield Banking Pro', a flagship mobile financial platform operating on Android and iOS. The objective of this evaluation was to rigorously map the application's attack surface, evaluate client-side binary protections, assess communication layer resilience, and execute realistic multi-stage exploitation scenarios to determine the business risk posed by sophisticated malicious actors.

The assessment identified a total of 26 unique security vulnerabilities spanning across client-side binary posture, local storage, inter-process communication (IPC), network traffic management, and server-side API endpoints. Notably, 3 Critical and 6 High severity vulnerabilities were uncovered.

KEY EXECUTIVE TAKEAWAY & RISK RATING: CRITICAL

The application relies heavily on client-side controls for security logic. Weak reverse engineering protections, combined with hardcoded cryptographic keys in native shared libraries (.so) and insufficient server-side authorization checks, allow an attacker to achieve Full Account Takeover (ATO) and unauthorized financial transaction execution.

2. Mobile Application Architecture & Assessment Scope

Mobile applications interact with complex ecosystem layers—including native device storage, runtime OS APIs, network channels, and cloud microservices. A thorough assessment requires dissecting each layer of the modern mobile application stack.

Layer / Domain	Target Component / Binary	Testing Environment	Assessment Focus Area
Binary & Client Layer	FinShield.apk / FinShield.ipa Native C++ Libs (libcore.so)	Rooted Pixel 6 (Android 14) Jailbroken iPhone 13 (iOS 16)	Decompilation, Code Obfuscation, Anti-Debugging, Static Secrets

Data Storage Layer	SharedPreferences, EncryptedSQLite, Android Keystore / iOS Keychain	Physical Access & ADB / File System Dumps	Plaintext Token Caching, Database Key Derivation, Logcat Leaks
Communication Layer	HTTPS/TLS 1.3 Endpoint Comms, WebSocket Feeds	Burp Suite Professional & Custom Certificate Injection	TLS Pinning Validation, MitM Interception, Replay Attacks
IPC & Platform Layer	Exported Activities, Receivers, Deep Links & WebViews	Frida 16.1.x, Drozer Framework, Activity Manager Injection	Intent Injection, Cross-App Data Leakage, JS Interface Flaws

3. Mobile Application Attack Surface Mapping

To establish a structured threat model, the threat surface of FinShield Banking Pro was mapped across four critical vectors.

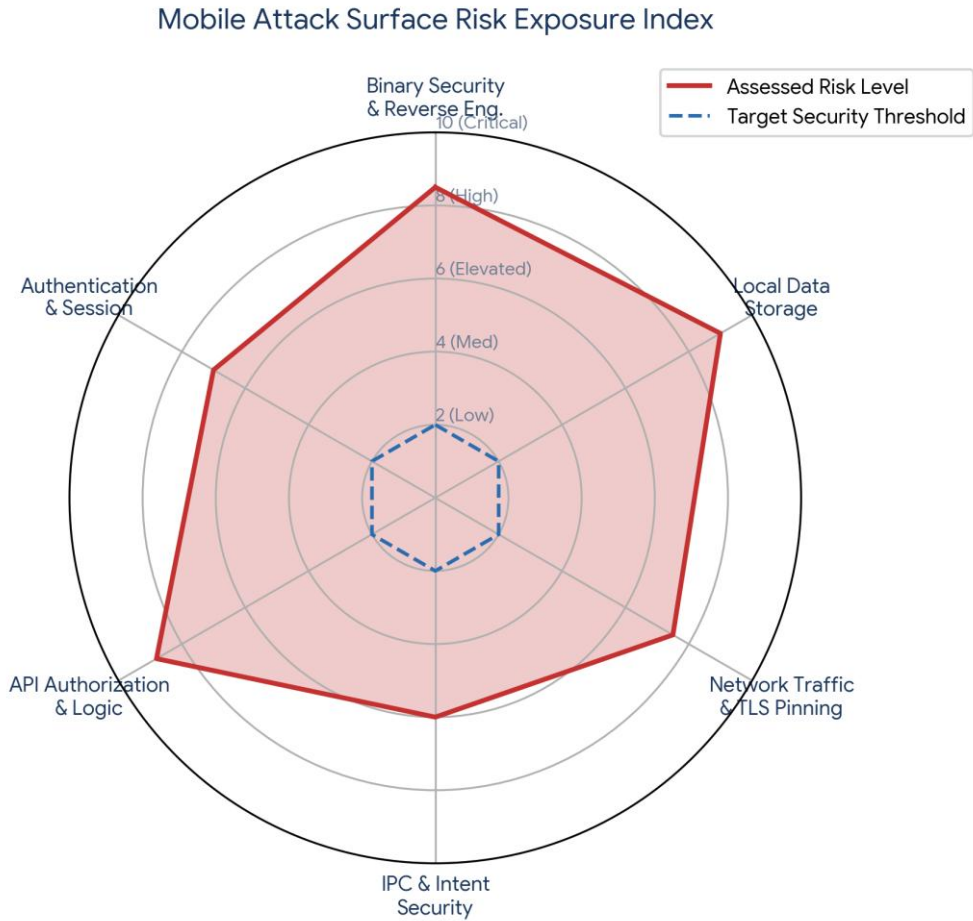


Figure 1: Mobile Application Attack Surface Risk Exposure Index across Core Vectors.

4. Deep-Dive Technical Exploitation Scenarios

To demonstrate the operational impact of the identified attack vectors, assessors executed end-to-end multi-stage exploitation scenarios.

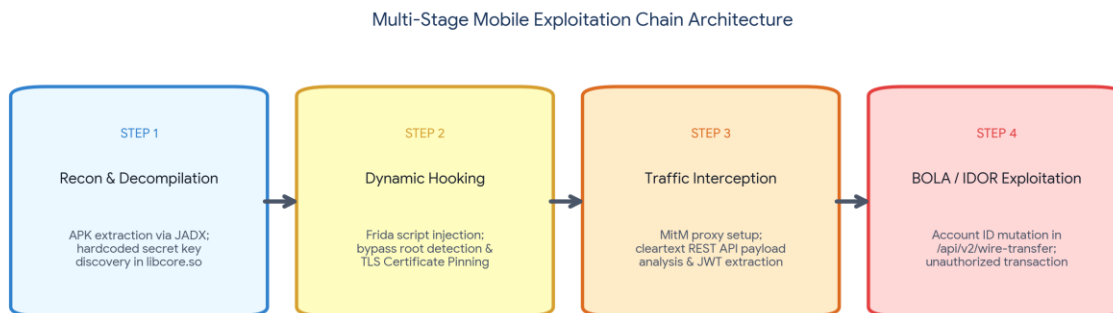


Figure 2: Multi-Stage Mobile Exploitation Chain Architecture Flow.

Scenario 1: Dynamic Instrumentation (Frida) to Defeat Root Detection & TLS Pinning

Assessors injected a custom Frida script into the running application process space to overwrite Java method return values and bypass trust managers:

```
// Frida Dynamic Hooking Script: Bypass Root Check & SSL Pinning
Java.perform(function () {
  console.log("[+] Intercepting FinShield Integrity Checks...");
  var DeviceIntegrity = Java.use("com.finshield.security.DeviceIntegrity");
  DeviceIntegrity.isDeviceRooted.implementation = function () {
    console.log("[*] Root detection check intercepted -> Forcing return: FALSE");
    return false;
  };
  var TrustManagerImpl = Java.use("com.android.org.conscrypt.TrustManagerImpl");
  TrustManagerImpl.verifyChain.implementation = function (untrustedChain, trustAnchorGroup,
    host, clientAuth, ocspData, tlsSslEngine) {
    return untrustedChain; // Return original chain without CertificateException
  };
});
```

Scenario 2: Broken Object Level Authorization (BOLA) API Exploitation

Modifying the `source\_account\_id` parameter in cleartext HTTPS REST requests allowed authorized transfers directly out of victim accounts:

```
POST /api/v2/wire-transfer HTTP/1.1
Host: api.finshield-bank.com
Authorization: Bearer eyJhbGciOiJIUzI1Ni... (User A Session Token)

{
  "source_account_id": "ACC-9940182", <-- Attacker changed to Victim Account ID
  "destination_account_id": "ACC-1002948",
  "amount": 5000.00
}
```

5. Assessment Findings & OWASP Mobile Top 10 Alignment

The identified vulnerabilities were compiled and categorized against the OWASP Mobile Top 10 framework:

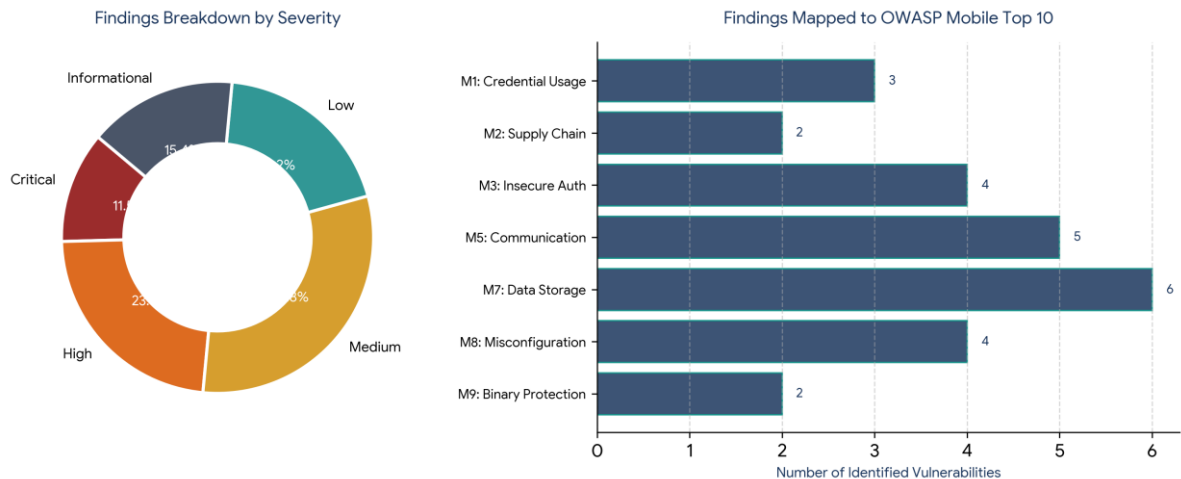


Figure 3: Vulnerability Severity Distribution & Mapping to OWASP Mobile Top 10.

ID	Vulnerability Title	OWASP Category	CVSS v3.1	Severity
SEC-01	Broken Object Level Authorization (BOLA) in Transaction API	M1: Credential Usage / API1	9.1	CRITICAL
SEC-02	Hardcoded Cryptographic Keys in Native Shared Object (.so)	M1: Credential Usage	8.6	CRITICAL
SEC-03	Arbitrary Intent Hijacking & Remote Data Exfiltration via WebView	M4: Input/Output Validation	8.2	HIGH
SEC-04	Insecure Local Storage & Key Derivation in SQLCipher	M7: Insecure Data Storage	7.8	HIGH
SEC-05	Client-Side Trust of TLS Pinning & Root Detection State	M5: Insecure Communication	7.1	HIGH

6. Strategic Conclusion & Sign-Off

Mobile applications must be designed under a Zero-Trust client paradigm: the client device must always be assumed compromised. Implementing robust server-side authorization checks, hardware-backed KeyStore encryption, and RASP controls is essential to mitigate these exploitation vectors.